



MetaDAO Futarchy Launchpad

Smart Contract Security Assessment

October 2025

Prepared for:

MetaDAO

Prepared by:

Offside Labs

Ripples Wen

Gongyu Shi





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Incorrect Refund Amount on Oversubscription	6
4.2	Mutual Exclusion: Claim vs Refund Blocks Partial Settlement	6
4.3	Missing Length Validation of monthly_spending_limit_members May Lead to DoS	7
4.4	Permissionless Claim Sends Rent To Caller	8
4.5	Incorrect Vault Invariant After Transfers	9
4.6	Inverted Time-Window Check in complete_launch	10
4.7	Excessive Premine Allowed Due to Incorrect Amount Check	11
4.8	Persistent FundingRecord Account Leading to Excess Rent Consumption	12
4.9	Informational and Undetermined Issues	12
5	Disclaimer	16



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs





2 Executive Summary

Introduction

Offside Labs completed a security audit of *MetaDAO Futarchy Launchpad* smart contracts, starting on September 24th, 2025, and concluding on October 1st, 2025.

Project Overview

MetaDAO Launchpad is a decentralized funding platform that enables projects to raise initial capital through community-driven token launches while establishing immediate on-chain governance. Most of the raised funds are transferred to the treasury of the newly formed DAO, which implements governance based on futarchy, allowing participants to trade on governance decisions.

Audit Scope

The assessment scope contains mainly the smart contracts of the launchpad program for the *MetaDAO Futarchy Launchpad* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- MetaDAO Futarchy Launchpad
 - Codebase: <https://github.com/metaDAOproject/programs>
 - PR-336
 - Commit Hash: 4c731a15b9237fa3d47677f7f23dedbb700ec154
 - Link: [PR-336](#)
 - PR-338
 - Commit Hash: 65edb88c80207cc70386786ae83b79c6f06994cd
 - Link: [PR-338](#)

We listed the files we have audited below:

- MetaDAO Futarchy Launchpad
 - programs/launchpad/src/instructions/claim.rs
 - programs/launchpad/src/instructions/close_launch.rs
 - programs/launchpad/src/instructions/complete_launch.rs
 - programs/launchpad/src/instructions/fund.rs
 - programs/launchpad/src/instructions/initialize_launch.rs
 - programs/launchpad/src/instructions/refund.rs
 - programs/launchpad/src/instructions/start_launch.rs
 - programs/launchpad/src/state/funding_record.rs
 - programs/launchpad/src/state/launch.rs
 - programs/launchpad/src/lib.rs



Findings

The security audit revealed:

- 2 critical issues
- 2 high issues
- 3 medium issues
- 1 low issue
- 7 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect Refund Amount on Oversubscription	Critical	Fixed
02	Mutual Exclusion: Claim vs Refund Blocks Partial Settlement	Critical	Fixed
03	Missing Length Validation of monthly_spending_limit_members May Lead to DoS	High	Fixed
04	Permissionless Claim Sends Rent To Caller	High	Fixed
05	Incorrect Vault Invariant After Transfers	Medium	Fixed
06	Inverted Time-Window Check in complete_launch	Medium	Fixed
07	Excessive Premine Allowed Due to Incorrect Amount Check	Medium	Fixed
08	Persistent FundingRecord Account Leading to Excess Rent Consumption	Low	Acknowledged
09	LaunchRefundedEvent Logs Incorrect Amount	Informational	Fixed
10	Unchecked final_raise_amount May Exceed Committed Amount	Informational	Fixed
11	Sequence Number of Funding Record Not Updated in claim and refund	Informational	Fixed
12	Unconditional SOL Transfer to Launch Signer	Informational	Acknowledged
13	Hard-Coded Magic Number Used for Futarchy Pool USDC Calculation	Informational	Acknowledged
14	Unimplemented Logic in damm_v2_cpi Program	Informational	Acknowledged
15	Redundant Check and Account Constraints	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Incorrect Refund Amount on Oversubscription

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When the launch is complete and oversubscribed, refund should return the unaccepted quote (USDC) to each participant:

- `expected_refund = committed_amount - accepted_quote`
- `accepted_quote = committed_amount × final_raise_amount / total_committed_amount`

The current implementation instead refunds the accepted portion, not the remainder.

Impact

As a result, users receive the accepted portion rather than the remaining unaccepted portion. This may lead to excessive refunds being issued, causing financial discrepancies and potential loss of funds within the system.

Recommendation

Modify the refund logic to return only the unaccepted remainder (`committed_amount - accepted_quote`) to each participant.

Mitigation Review Log

Fixed in commit 1c03b9494d22d695ca18bf8b56cb66b4c7e84706.

4.2 Mutual Exclusion: Claim vs Refund Blocks Partial Settlement

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In oversubscription scenarios (`final_raise_amount < total_committed_amount`), participants must both:



- Claim base tokens for the accepted portion
- Receive a refund of the unaccepted quote (USDC)

However, the instruction design prevents performing both operations:

- Claim closes the funding record, blocking any subsequent refund
- Refund zeroes the committed amount in funding record, blocking any subsequent claim yield zero tokens

Impact

Users cannot fully settle their position post-oversubscription, results in a loss of funds or tokens for affected users and inconsistent token/quote accounting.

Recommendation

It is recommended to correct the logic so that the `FundingRecord` can only be closed once both the claim and refund operations have been completed in the oversubscription scenario mentioned above.

Mitigation Review Log

Fixed in commit `ad43b8683f2f3f66a70ee1ac28924b9b9aed0bdc` by preventing the `FundingRecord` from being closed in either instruction.

4.3 Missing Length Validation of `monthly_spending_limit_members` May Lead to DoS

Severity: High

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

When initializing the launch account, the passed-in vector `monthly_spending_limit_members` is assigned directly to the corresponding field without any length check. The expected maximum length of this vector is 10.

```
36     #[max_len(10)]
37     pub monthly_spending_limit_members: Vec<Pubkey>,
38     ...
39     /// The DAO, if the launch is complete.
40     pub dao: Option<Pubkey>,
41     /// The DAO treasury that USDC / LP is sent to, if the launch is
42     /// complete.
43     pub dao_vault: Option<Pubkey>,
```



[programs/launchpad/src/state/launch.rs#L36-L42](#)

Without this check, a vector longer than 10 can be passed in, which causes the reserved space for `dao` and `dao_vault` to be overwritten. This does not result in an immediate DoS, but it prevents `complete_launch` from executing successfully, since it fails when attempting to initialize the `dao` and `dao_vault`.

Impact

If the length of vector `monthly_spending_limit_members` is greater than 10, and the committed amount meets the minimum raise requirement, then the launch can never be completed. As a result, users may not claim the tokens as well as the refunded quote tokens.

Recommendation

It is recommended to add an explicit check in `initialize_launch` to ensure the length of `monthly_spending_limit_members` cannot exceed 10.

Mitigation Review Log

Fixed in commit 416292f4281f96e0cbf10086a475b51551a5e340.

4.4 Permissionless Claim Sends Rent To Caller

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The claim instruction is permissionless. Anyone can trigger a claim for any funder because the funder account is not required to be a signer. While base tokens are transferred to the rightful funder's token account, the `FundingRecord` is closed to the payer, sending the rent exempt lamports to the caller rather than the funder.

```
20     #[account(  
21         mut,  
22         close = payer,  
23         has_one = funder,  
24         seeds = [b"funding_record", launch.key().as_ref(),  
25         funder.key().as_ref()],  
26         bump = funding_record.pda_bump  
27     )]  
28     pub funding_record: Account<'info, FundingRecord>
```

[programs/launchpad/src/instructions/claim.rs#L20-L27](#)



Impact

Third parties can mass claim for others to harvest the rent from closing each `FundingRecord`. While tokens go to the correct funder, the lamports do not.

Recommendation

Refund the lamports to the funder instead.

Mitigation Review Log

Fixed in commit `ad43b8683f2f3f66a70ee1ac28924b9b9aed0bdc` by preventing the `FundingRecord` from being closed in this case.

4.5 Incorrect Vault Invariant After Transfers

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

At the end of `complete_launch`, the code verifies that then number of the USDC tokens in the quote vault is not less than `usdc_to_dao`.

```
705     self.launch_quote_vault.reload()?;  
706     ...  
707     require_gte!(  
708         self.launch_quote_vault.amount,  
709         usdc_to_dao,  
710         LaunchpadError::InvariantViolated  
711     );
```

[programs/launchpad/src/instructions/complete_launch.rs#L705-L711](#)

This invariant is checked after transferring both liquidity `usdc_to_lp` and DAO funds `usdc_to_dao` out of the quote vault and reloading the vault account, so the remaining balance is typically `total_committed_amount - final_raise_amount`, which is likely to be less than `usdc_to_dao`. Consequently, the invariant check fails in common scenarios.

Impact

Systematic transaction reverts for `complete_launch` under typical oversubscription ratios, causing a denial-of-service on launch finalization and potentially trapping funds in the launch vaults.



Recommendation

It is recommended to revise the validation logic to verify the correct post-transfer invariant.

Mitigation Review Log

Fixed in commit 84d24fc6b32066b5294c251d952abf838a551860.

4.6 Inverted Time-Window Check in `complete_launch`

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

```
164 // if the launch was closed within 2 days, the launch authority must be
    ↳ the one
165 // to complete the launch
166 if self.launch.unix_timestamp_closed.unwrap() + 60 * 60 * 24 * 2 <
    ↳ clock.unix_timestamp {
167     require!(self.launch_authority.is_some(),
    ↳ LaunchpadError::LaunchAuthorityNotSet);
168
169     require_keys_eq!(self.launch_authority.as_ref().unwrap().key(),
    ↳ self.launch.launch_authority,
    ↳ LaunchpadError::LaunchAuthorityNotSet);
170 }
```

[programs/launchpad/src/instructions/complete_launch.rs#L164-L170](#)

The authorization check in `complete_launch` is inverted relative to the comment/intended design. The code enforces that only the launch authority may complete the launch after two days have elapsed since closure, rather than within the first two days. This reversal permits permissionless completion during the first 48 hours and restricts completion after 48 hours to the authority only.

Impact

Any caller can complete the launch within the first 48 hours after closing, bypassing the intended authority gate during the critical post-closure period.

Recommendation

Invert the time comparison to gate within the window, not after it.



Mitigation Review Log

Fixed in commit `ad43b8683f2f3f66a70ee1ac28924b9b9aed0bdc`.

4.7 Excessive Premine Allowed Due to Incorrect Amount Check

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When initializing the launch, a check is added to restrict the `performance_package_token_amount`, which represents the amount of locked tokens:

```
122     require_gte!(  
123         MAX_PREMINE * TOKEN_SCALE,  
124         args.performance_package_token_amount,  
125         LaunchpadError::InvalidPriceBasedPremineAmount  
126     );
```

[programs/launchpad/src/instructions/initialize_launch.rs#L122-L126](#)

```
39  /// Max 50% premine  
40  pub const MAX_PREMINE: u64 = 10_000_000 * TOKEN_SCALE;
```

[programs/launchpad/src/lib.rs#L39-L40](#)

However, `TOKEN_SCALE` is applied twice, which allows a much larger premine than intended.

Impact

The maximum number of locked tokens should be 10 million (10^7 without `TOKEN_SCALE`), but due to the redundant scaling, it can instead reach 10 trillion (10^{13} without `TOKEN_SCALE`). This could cause the tokenomics to deviate significantly from the intended design.

Recommendation

It is recommended to correct the logic by removing the redundant `TOKEN_SCALE` in the check.

Mitigation Review Log

Fixed in commit `44afca9d3fe97b461e1a50c370be28b6c4114889`.



4.8 Persistent FundingRecord Account Leading to Excess Rent Consumption

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

Description

The `FundingRecord` account is not properly closed after both the claim and refund operations. As a result, the account continues to occupy rent-exempt storage space, leading to unnecessary rent consumption and inefficient resource utilization.

Impact

Failing to close the `FundingRecord` account after its lifecycle increases operational costs.

Recommendation

It is recommended to implement explicit logic to close the `FundingRecord` account immediately after both claim and refund operations are completed, or alternatively, to merge the `claim` and `refund` IXs into a single `claim_and_refund` IX.

Mitigation Review Log

MetaDAO Team: I'm fine with this for now, I think long-term it would make sense to do the `claim_and_refund` thing.

4.9 Informational and Undetermined Issues

LaunchRefundedEvent Logs Incorrect Amount

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

In the refund instruction, the emitted `LaunchRefundedEvent` event field `usdc_refunded` currently records the value of `funding_record.committed_amount` rather than the actual refunded amount, represented by `amount_to_refund`. Furthermore, since `committed_amount` is reset to zero immediately after computing `amount_to_refund`, the emitted event will always report a value of zero.

Fixed in commit 23f07f377941bc1f407d3abaf10564bb721ec5bf.



Unchecked final_raise_amount May Exceed Committed Amount

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In `complete_launch`, the `final_raise_amount` is not checked against `launch.total_committed_amount`. In certain edge cases, for example, if someone mistakenly or deliberately transfers extra USDC into the `launch_quote_vault` directly, it is possible for `final_raise_amount` to exceed `launch.total_committed_amount`.

Fixed in commit 7554aa03a673c8f63bf9ff7fdc3b6224cd60075f.

Sequence Number of Funding Record Not Updated in claim and refund

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

The `seq_num` field of `FundingRecord` is incremented by one for each `fund` operation. However, this field is not updated when base tokens are claimed in `claim` or when quote tokens are refunded in `refund`.

Fixed in commit 7b824a431882d666b4158e4e6a7fbe825d803648 by removing this field.

Unconditional SOL Transfer to Launch Signer

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Optimization

When completing the launch, 0.05 SOL is transferred from the `payer` to the `launch_signer`, regardless of whether the `launch_signer` already holds some lamports. It is recommended to add a check so that only the necessary amount of lamports is transferred.

```
518     system_program::transfer(  
519         CpiContext::new(  
520             ctx.accounts.system_program.to_account_info(),  
521             system_program::Transfer {  
522                 from: ctx.accounts.payer.to_account_info(),  
523                 to: ctx.accounts.launch_signer.to_account_info(),  
524             },  
525         ),  
526         50_000_000,  
527     )?;
```

[programs/launchpad/src/instructions/complete_launch.rs#L518-L527](#)



Hard-Coded Magic Number Used for Futarchy Pool USDC Calculation

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code QA

When calculating the portion of USDC allocated to provide liquidity, a hard-coded value `5` is used. This can be replaced with a calculation based on token distribution `final_raise_amount * TOKENS_TO_FUTARCHY_LIQUIDITY / TOKENS_TO_PARTICIPANTS`.

```
305 let usdc_to_lp = final_raise_amount.saturating_div(5);  
programs/launchpad/src/instructions/complete\_launch.rs#L305-L305
```

Unimplemented Logic in `damm_v2_cpi` Program

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

Currently, the `damm_v2_cpi` program serves only as a mock implementation. The `initialize_pool_with_dynamic_config` instruction performs no actions, and none of the passed-in accounts are validated.

Redundant Check and Account Constraints

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code QA

We identified several redundant validations and constraints in `complete_launch`:

- The program explicitly validates the decimals of `base_mint` and `quote_mint`.

```
529 require_eq!(  
530     ctx.accounts.base_mint.decimals,  
531     6,  
532     LaunchpadError::InvariantViolated  
533 );  
534 require_eq!(  
535     ctx.accounts.quote_mint.decimals,  
536     6,  
537     LaunchpadError::InvariantViolated  
538 );
```

[programs/launchpad/src/instructions/complete_launch.rs#L529-L538](#)

Note that these fields are immutable and were already checked during `initialize_launch`.

- Constraints on `launch_quote_vault` and `launch_base_vault` validate their mint and authority, but both accounts are already validated via the `has_one` constraint on the



launch account:

```
138 #[account(  
139     mut,  
140     has_one = launch_quote_vault,  
141     has_one = launch_base_vault,  
142     ...  
143 )]  
144 pub launch: Box<Account<'info, Launch>>,
```

[programs/launchpad/src/instructions/complete_launch.rs#L138-L144](#)

- Several accounts (`squads_program_config` , `dao_owned_lp_position` , `squads_multisig` , `squads_multisig_vault` , `spending_limit`) will be checked in the `initialize_dao` IX by the `futarchy` program. The same constraints are redundantly applied in the `launchpad` program.

```
213 /// CHECK: checked by autocrat program  
214 #[account(mut, seeds = [squads_multisig_program::SEED_PREFIX,  
    ↪ squads_multisig_program::SEED_MULTISIG, dao.key().as_ref()], bump,  
    ↪ seeds::program = static_accounts.squads_program)]  
215 pub squads_multisig: UncheckedAccount<'info>,  
216 /// CHECK: just a signer  
217 #[account(seeds = [squads_multisig_program::SEED_PREFIX,  
    ↪ squads_multisig.key().as_ref(), squads_multisig_program::SEED_VAULT,  
    ↪ 0_u8.to_le_bytes().as_ref()], bump, seeds::program =  
    ↪ static_accounts.squads_program)]  
218 pub squads_multisig_vault: UncheckedAccount<'info>,  
219 /// CHECK: initialized by squads  
220 #[account(mut, seeds = [squads_multisig_program::SEED_PREFIX,  
    ↪ squads_multisig.key().as_ref(),  
    ↪ squads_multisig_program::SEED_SPENDING_LIMIT, dao.key().as_ref()],  
    ↪ bump, seeds::program = static_accounts.squads_program)]  
221 pub spending_limit: UncheckedAccount<'info>,
```

[programs/launchpad/src/instructions/complete_launch.rs#L213-L221](#)



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

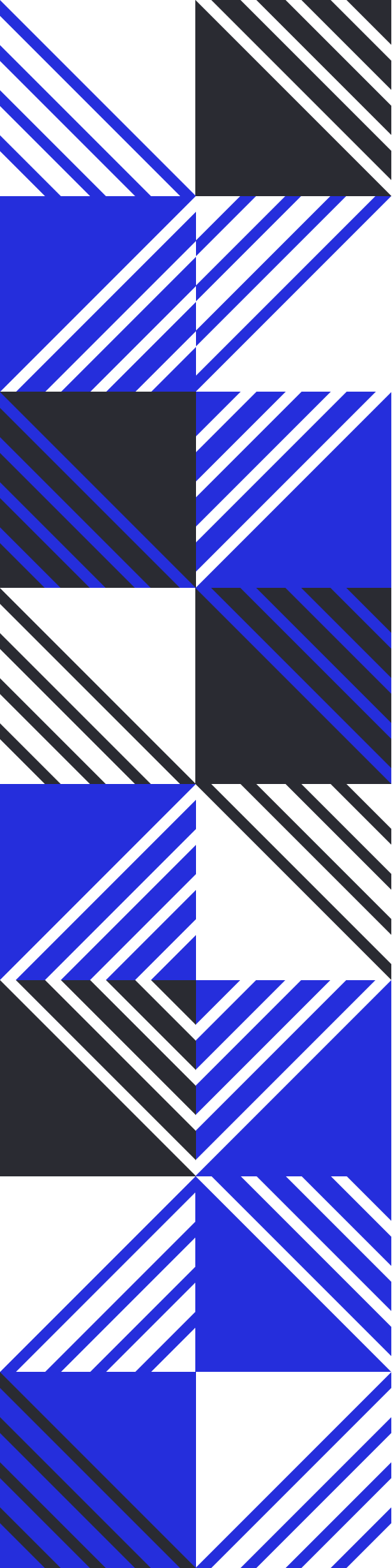
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs